Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение высшего образования «Томский государственный университет систем управления и радиоэлектроники»

Факультет систем управления (ФСУ)

Кафедра автоматизированных систем управления (АСУ)

Отчет по лабораторной работе №3

по дисциплине «Современные средства программирования»

Обучающийся гр. 435-М1

_____Андреев Д.П.

«_____»_____2025 г.

Проверил: доцент каф. АСУ, к.т.н

_____Алфёров С.М,

«___»_____2025 г.

Томск 2025

1. Цель работы.

Получить навыки создания иерархии классов с использованием абстрактных классов; уяснить пользу от использования абстрактных классов.

2. Индивидуальное задание

Создать указанный абстрактный класс. Создать несколько производных классов, реализующие поведение абстрактного класса. Продемонстрировать возможность работы с объектами разных классов через методы абстрактного класса.

Вариант 2:

Двигатель Вход: f – требуемая частота вращения двигателя Выход: драйвер должен сформировать управляющие сигналы	1. Шаговый Для вращения с определенной частотой необходимо передавать в порт последовательность байтов с соответствующим временным шагом (4 старших бита не используются): 00000001b, 00000010b, 00000100b, 00001000b. Для вращения в обратном направлении байты передавать в обратном порядке. Чтобы сымитировать выдачу сигналов с определенным временным шагом, вместе с байтом можно отобразить временную метку. 2. Постоянного тока Для вращения с заданной частотой необходимо однократно передать в контроллер последовательность: код символа 'u' (байт), значение напряжения в В (float). Для подтверждения задания частоты принять от контроллера сообщение: код символа 'f' (байт), значение частоты вращения в Гц (float). Для вращения в обратном направлении передавать значение напряжения с противоположным знаком.

3. Решение

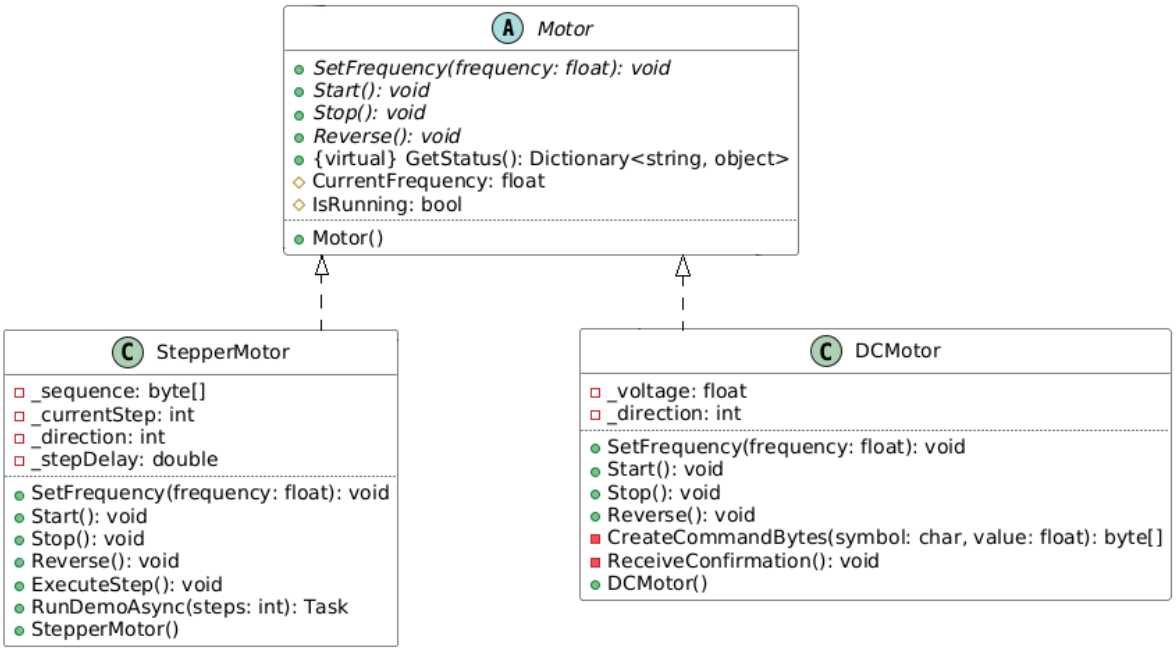

Рисунок 3.1 – UML-диаграмма классов

Листинг программы:

Program.cs:

```
using System;
using System.Collections.Generic;

using System.Threading.Tasks;

namespace ССП3
{
    public class MotorDemo
    {
        public static void DemonstrateMotors()
        {
            // Создаем двигатели
            var stepper = new StepperMotor();
            var dcMotor = new DCMotor();

            var motors = new List<Motor> { stepper, dcMotor };

            Console.WriteLine(new string('=', 50));
            Console.WriteLine("ДЕМОНСТРАЦИЯ РАБОТЫ С ДВИГАТЕЛЯМИ");
            Console.WriteLine(new string('=', 50));

            // Демонстрируем работу с каждым двигателем через методы абстрактного класса
            for (int i = 0; i < motors.Count; i++)
            {
                var motor = motors[i];
                Console.WriteLine($"\n--- Двигатель {i + 1}: {motor.GetType().Name} ---");

                // Получаем начальный статус
                Console.WriteLine("Начальный статус:");
                PrintStatus(motor.GetStatus());
```

```
            // Устанавливаем частоту
            motor.SetFrequency(2.0f); // 2 Гц

            // Изменяем направление
            motor.Reverse();

            // Снова устанавливаем частоту
            motor.SetFrequency(3.0f);

            // Получаем финальный статус
            Console.WriteLine("Финальный статус:");
            PrintStatus(motor.GetStatus());

            // Останавливаем двигатель
            motor.Stop();
        }
    }

    private static void PrintStatus(Dictionary<string, object> status)
    {
        foreach (var item in status)
        {
            Console.WriteLine($"  {item.Key}: {item.Value}");
        }
    }
  }

  class Program
  {
    static void Main(string[] args)
    {
            MotorDemo.DemonstrateMotors();
            Console.ReadKey();
    }
  }
}
```

Motor.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ССП3
{
  public abstract class Motor
  {
    public float Frequency { get; protected set; }//Частота
    public bool IsRunning { get; protected set; }

    protected Motor()
    {
        Frequency = 0;
        IsRunning = false;
    }

    public abstract void SetFrequency(float frequency);
    public abstract void Start();
    public abstract void Stop();
    public abstract void Reverse();
```

```
        public virtual Dictionary<string, object> GetStatus()
        {
            return new Dictionary<string, object>
        {
            { "type", GetType().Name },
            { "current_frequency", Frequency },
            { "is_running", IsRunning }
        };
        }
    }
}
```

StepperMotor.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace ССП3
{
    public class StepperMotor : Motor// Шаговый двигатель
    {
        private readonly byte[] _sequence =
        {
        0b00000001,  // 1
        0b00000010,  // 2
        0b00000100,  // 4
        0b00001000   // 8
    };

        private int _currentStep; //текущий шаг
        private int _direction; // направление( 1 - вперед, -1 - назад)
        private double _stepDelay; // задержка между шагами в секундах

        public StepperMotor() : base()
        {
            _currentStep = 0;
            _direction = 1;
            _stepDelay = 0.1;
        }

        public override void SetFrequency(float frequency)
        {
            if (frequency == 0)
            {
                Stop();
                return;
            }

            Frequency = frequency;
            // Рассчитываем задержку между шагами на основе частоты
            // 1 Гц ≈ 1 шаг в секунду
            _stepDelay = 1.0 / (frequency * 4); // 4 шага на полный цикл

            if (!IsRunning)
            {
                Start();
            }

            Console.WriteLine($"Шаговый двигатель: установлена частота {frequency} Гц");
```

```
            Console.WriteLine($"Задержка между шагами: {_stepDelay:F4} сек");
        }

        public override void Start()
        {
            IsRunning = true;
            Console.WriteLine("Шаговый двигатель: запущен");
        }

        public override void Stop()
        {
            IsRunning = false;
            Frequency = 0;
            Console.WriteLine("Шаговый двигатель: остановлен");
        }

        public override void Reverse()
        {
            _direction *= -1;
            string directionText = _direction == 1 ? "прямое" : "обратное";
            Console.WriteLine($"Шаговый двигатель: направление изменено на {directionText}");
        }
    }
}
```

DCMotor.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace ССП3
{
    public class DCMotor : Motor// Двигатель постоянного тока
    {
        private float _voltage; // Напряжение
        private int _direction; // направление (1 - вперед, -1 - назад)

        public DCMotor() : base()
        {
            _voltage = 0.0f;
            _direction = 1;
        }

        public override void SetFrequency(float frequency)
        {
            Frequency = frequency;

            _voltage = frequency * 0.1f * _direction; // 0.1 В на 1 Гц

            // Формируем последовательность для отправки в контроллер
            // код символа 'u' + значение напряжения (float)
            byte[] voltageBytes = CreateCommandBytes('u', _voltage);

            Console.WriteLine("Двигатель постоянного тока: отправлена команда:");
            Console.WriteLine($"  Код символа: 'u' ({(int)'u'})");
            Console.WriteLine($"  Напряжение: {_voltage:F2} В");
            Console.WriteLine($"  Байты: {BitConverter.ToString(voltageBytes)}");

            // Имитация получения подтверждения от контроллера
```

```
        ReceiveConfirmation();

        if (!IsRunning && frequency != 0)
        {
            Start();
        }
        else if (frequency == 0)
        {
            Stop();
        }
    }

    private byte[] CreateCommandBytes(char symbol, float value)
    {
        byte[] bytes = new byte[5]; // 1 байт для символа + 4 байта для float
        bytes[0] = (byte)symbol;
        byte[] floatBytes = BitConverter.GetBytes(value);
        Array.Copy(floatBytes, 0, bytes, 1, 4);
        return bytes;
    }

    private void ReceiveConfirmation()
    {
        // Имитируем задержку обработки
        Thread.Sleep(100);

        // Формируем подтверждение: код 'f' + частота (float)
        byte[] confirmationBytes = CreateCommandBytes('f', Frequency);

        Console.WriteLine("Двигатель постоянного тока: получено подтверждение:");
        Console.WriteLine($"  Код символа: 'f' ({(int)'f'})");
        Console.WriteLine($"  Частота: {Frequency:F2} Гц");
        Console.WriteLine($"  Байты: {BitConverter.ToString(confirmationBytes)}");
    }

    public override void Start()
    {
        IsRunning = true;
        Console.WriteLine("Двигатель постоянного тока: запущен");
    }

    public override void Stop()
    {
        IsRunning = false;
        Frequency = 0;
        _voltage = 0.0f;
        Console.WriteLine("Двигатель постоянного тока: остановлен");
    }

    public override void Reverse()
    {
        _direction *= -1;
        string directionText = _direction == 1 ? "прямое" : "обратное";
        Console.WriteLine($"Двигатель постоянного тока: направление изменено на {directionText}");

        // При изменении направления нужно обновить напряжение
        if (IsRunning)
        {
            SetFrequency(Frequency);
        }
    }
}}
```

4. Результаты работы

Демонстрация работы шагового двигателя представлена на рисунке 4.1

```
==================================================
ДЕМОНСТРАЦИЯ РАБОТЫ С ДВИГАТЕЛЯМИ
==================================================

--- Двигатель 1: StepperMotor ---
Начальный статус:
  type: StepperMotor
  current_frequency: 0
  is_running: False
Шаговый двигатель: запущен
Введите кол. шагов:4

Демонстрация шагового двигателя (4 шагов):
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000001
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000010
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000100
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00001000
Шаговый двигатель: установлена частота 2 Гц
Задержка между шагами: 0,1250 сек
Шаговый двигатель: направление изменено на обратное
Введите кол. шагов:5

Демонстрация шагового двигателя (5 шагов):
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000001
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00001000
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000100
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000010
[25.11.2025 20:21:04] Шаговый двигатель: отправлен байт 00000001
Шаговый двигатель: установлена частота 3 Гц
Задержка между шагами: 0,0833 сек
Финальный статус:
  type: StepperMotor
  current_frequency: 3
  is_running: True
Шаговый двигатель: остановлен
```

Рисунок 4.1 – Демонстрация работы шагового двигателя

Далее демонстрация работы электрического двигателя на рисунке 4.2.

```
--- Двигатель 2: DCMotor ---
Начальный статус:
  type: DCMotor
  current_frequency: 0
  is_running: False
Двигатель постоянного тока: отправлена команда:
  Код символа: 'u' (117)
  Напряжение: 0,20 В
  Байты: 75-CD-CC-4C-3E
Двигатель постоянного тока: получено подтверждение:
  Код символа: 'f' (102)
  Частота: 2,00 Гц
  Байты: 66-00-00-00-40
Двигатель постоянного тока: запущен
Двигатель постоянного тока: направление изменено на обратное
Двигатель постоянного тока: отправлена команда:
  Код символа: 'u' (117)
  Напряжение: -0,20 В
  Байты: 75-CD-CC-4C-BE
Двигатель постоянного тока: получено подтверждение:
  Код символа: 'f' (102)
  Частота: 2,00 Гц
  Байты: 66-00-00-00-40
Двигатель постоянного тока: отправлена команда:
  Код символа: 'u' (117)
  Напряжение: -0,30 В
  Байты: 75-9A-99-99-BE
Двигатель постоянного тока: получено подтверждение:
  Код символа: 'f' (102)
  Частота: 3,00 Гц
  Байты: 66-00-00-40-40
Финальный статус:
  type: DCMotor
  current_frequency: 3
  is_running: True
Двигатель постоянного тока: остановлен
```

Рисунок 4.2 – Демонстрация работы электрического двигателя

5. Вывод

В результате выполнения лабораторной работы я получил навык создания иерархии классов с использованием абстрактных классов. Была изучена польза от использования абстрактных классов.